QUANSER
INNOVATE·EDUCATE

Lab Procedure
Brushless DC (BLDC) Motor

Introduction

Ensure the following:

1. You have reviewed the **Application Guide – BLDC**
2. Make sure you have the provided **BLDC motor**.
3. Make sure the **Mechatronic Actuators Trainer** is out of its case; it is powered using the provided power supply and connected to the PC.
    a. Turn it on using the Power switch on the side of the device.
    b. Both the USB and Power LED should be green.

This experiment consists of two parts that will help you understand how a BLDC motor works and how to drive it. First, you will explore how the motor works, how the magnets and the sensors inside interact. The second section of the lab will walk you through using this information to learn how to drive the motor based on its sensor outputs.

Hall Sensor Exploration

In the case of the motor we are using, the datasheet does not give any information about hall sensor placement on the motor, specifically where A, B and C are in relation to each other. This section explores how to figure out where each sensor is placed. As well, it explores how the movement of the magnet affects the hall sensor readings and how signal cycles are related to pole pairs of the motor.

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**..., and browse to the directory containing the python files and lab procedure for the BLDC motor lab (…/actuators_trainer/1_fundamentals/brushless_dc/hardware/python). Open this directory.
2. Open **bldc_hall.py** and locate the following line:

    ```
    with ActuatorsTrainer(block = 2) as actuators:
    ```

    This line is present in all labs and is used to initialize the device. The parameter **`block = 2`** dictates what block, as printed on the device, you are going to use. You can update this parameter if you want to connect the motors to a different block.
3. Connect your motor to the block you will be using.
    a. Use the **`Brushless DC`** connection for the motor power input.
    b. Use the **`Encoder`** connection for the hall sensor outputs. Make sure the black wire in the encoder cable matches the (–) sign labeled in the device.

4. Expected and 'normal' BLDC usage has 2 coils enabled at a time (one as high and one as ground) and one disabled (floating), so the current goes in through the 'high' one and exits to the one set as ground. Unlike this normal usage, to find where each hall sensor is located, we will enable all 3 and only set 1 to high, which means the other 2 will be set as ground (the current output will be split between the two ground phases). The file **bldc_hall.py** will set one coil as high at a time every 3 seconds and print the hall effect sensor reading once a second. We will refer to the 3 coils (sometimes referred to as phases) as A, B and C, and the Hall sensors as 1, 2 and 3.

5. As noted in the concept reviews, the hall sensor outputs a 1 when they detect a north pole in the magnet close by. When energizing a coil, a magnetic field is generated. In this special case where 3 coils are enabled, 1 as high and 2 as low, the magnetic field generated is as shown with a white arrow in Figure 1, which would move the magnet into the positions shown in the image. Note that the figure has coils named A, B, and C, and the sensors are denoted with shapes: a circle, triangle and square.

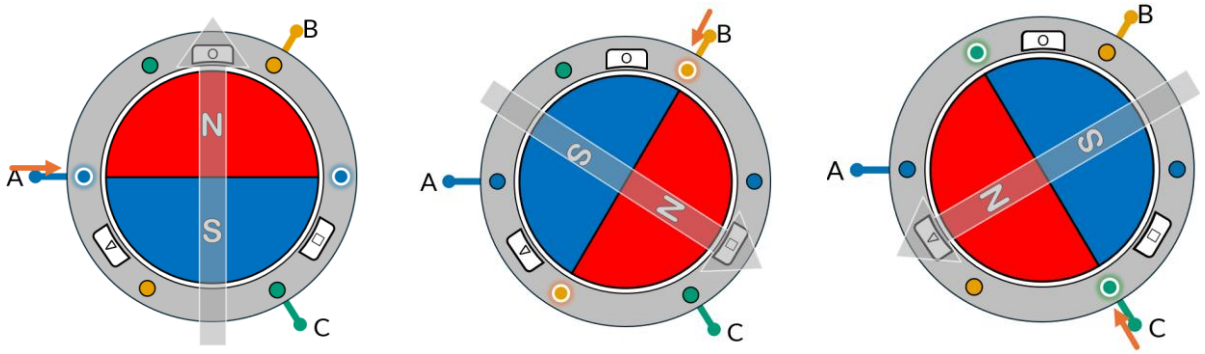


Figure 1. Magnet positions and magnetic fields when energizing a single coils with all 3 enabled

Note that the figure above is a simplification of the actual insides of a BLDC motor, however, it can be used to understand how things should work, however, energizing the A, B, and C phases in order will not constitute a full rotation of the motor shaft; this will be explored later in the lab. See the Concept Review mentioned in the Application Guide for further information.

6. Run **bldc_hall.py**, it will run for 30 seconds, look at the printed output, note what hall sensor goes high when each of the coils is activated. Note that the motor might feel like it changed and the print might happen a second later. It is expected. Based on that, where would you place sensor 1, 2 and 3 in Figure 1? Which one would be the circle, triangle and square?
   NOTE: Based on the way the code is set up, you will feel the motor move before the print output. It is okay, focus on the printed outputs in the terminal.
   NOTE: If at any BLDC command you see two hall sensors as 1, this could mean the command (amplitude) was too low to move the motor into the correct position, you can increase bldcAmplitude to 0.3 for example.

7. In what sequence would the hall sensors be activated in if the motor/magnet was rotating clockwise? Note that when the north side of the magnet is towards two hall sensors, both of them are high.

8. To aid yourself, make the following changes to your code:

a. Comment out the following two lines at the bottom of your code to not activate any coil. (By adding a # before each of the lines)

```
actuators.enable_bldc([1, 1, 1])
actuators.update_bldc(bldcCmd*bldcAmplitude)
```

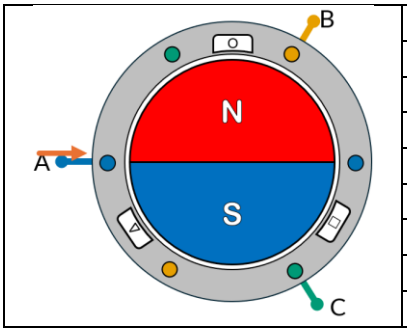
b. Change the line **`if cntr1s == frequency:`** to **`if cntr1s == 1:`** to print on every iteration of the program.

c. Modify the code to print only when there is a change in **`hallSensors`**.

d. Run the code again, rotate the motor clockwise *for one rotation* and then stop the code using **`Ctrl+C`** in the terminal. Stopping the code will print the encoder counts of the system in the terminal. The encoder counts value is based only on the A and B pulses of the hall sensors, the outputs from the C encoder do not get used in this calculation.

e. Observe the Hall Sensor logic printed on the console as well as the number of encoder counts.

9. Fill out the following table with the order of the hall sensor readings when moving the motor shaft (and therefore the magnet) clockwise. Make sure that your first entry on the table matches the position shown in the figure next to the table where the hall sensor that is high is the sensor marked with a circle now that you know where each sensor is located. Use 0's for when the Hall sensor is not active and 1's for when it is high. Note that you might not need to use all the available rows in the table.



Hall Sensor 1	Hall Sensor 2	Hall Sensor 3

10. How many different states does the hall sensor goes through for a rotation of the magnet? How many times did that sequence repeat itself when rotating the motor manually for one full rotation? Every rotor pole pair requires one signal cycle for one mechanical rotation. So, the number of signal cycles is equal to the rotor pole pairs. Compare this against the Number of Poles shown in the motor datasheet located in the brushless_dc folder.

11. How many encoder counts were read in 1 full mechanical rotation of the motor? The reason why this number is different from the amount of hall sensor states that happens in one full rotation is because only 2 of the 3 hall sensor outputs are used when reporting encoder counts. See the Concept Review on rotational sensors mentioned in the Application Guide for further information.

12. Notice the sign of the encoder when rotating clockwise vs counterclockwise. Run the code again if you need to.

13. How would you convert the encoder output from counts to radians?

14. Take each row of the hall sensor table you filled above as if it was a binary number and convert it into a decimal number, assume Hall Sensor 1 is the most significant bit. What order does it have for clockwise rotation?

Driving a BLDC Motor

Now that we have the order hall sensors get triggered in if the motor was rotating clockwise, we will use this to figure out which phases we need to energize to get the motor to rotate as expected. This section will explore how a BLDC motor is driven using hall sensors and how to program the device to spin the motor and change its direction and speed.

15. Determine the control sequence to rotate the motor clockwise. The table will be filled out in the next step. Note that the magnetic field should always be 90 degrees from the magnet to achieve maximum torque as shown in Figure 2. Figure 3 shows magnetic fields in different scenarios of voltage going in and out through 2 phases while the third phase without an arrow is left floating or not connected. For example, the first image shows current going in through coil/phase C (high), out through coil/phase A (low) and B left floating. Neither the magnet position nor the hall sensor location are relevant in this figure. See the Concept Review on BLDCs mentioned in the Application Guide for further information.

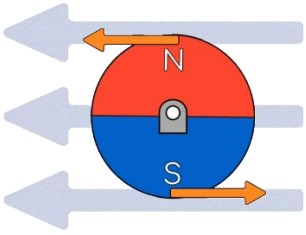


Figure 2. Magnetic Field for Maximum Torque

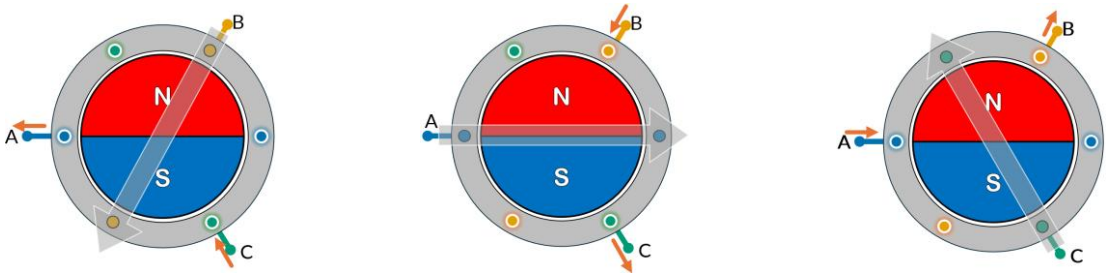


Figure 3. Magnetic fields generated when energizing coils

16. Note that the Quanser standard for encoder readings define that for a counterclockwise rotation, the encoder counts will go up. Since we are defining the order of the coil activation for a clockwise rotation, for a positive input, we will get clockwise rotation and therefore negative speeds.

17. Fill out the Hall Sensor states and the decimal representation in the first 4 columns which you have done in previous steps. Based on the sensor states in the table, fill out the control sequence needed to go to move the rotor/magnet to achieve the next sensor state. For each hall sensor state, note what phase(coil) should be High (voltage in), Low (voltage out) and Off (floating), using 1 for High, -1 for Low and 0 for Off.

Hall Sensor 1	Hall Sensor 2	Hall Sensor 3	Hall Sensors binary - decimal	Coil A	Coil B	Coil C

18. Open **bldc.py** and update the initialization line with the block you will use and connect the BLDC motor and the encoder to the device. This is described above, in the first 2 steps of the Hall Sensor Exploration section of this lab.

19. In the experiment constants section, locate where the array **`bldc`** is initialized with all 0's. Using the Phase A, B and C columns from the above table, update the values in the array, each line corresponds to one row in your table. Use 1 for High, -1 for Low and 0 for Off.

20. Make sure the **`bldcAmplitude`** is set to 0.15.

21. Locate the following comment in the code: **`# convert from binary to decimal`**, underneath, you should see **`hallState = 0`**, update that assignment so that the hallState variable converts the readings from the hall sensors into a decimal value as you did in the table above.
    NOTE: The variable hallSensors has 3 elements: [sensor1, sensor 2, sensor3].

22. Now that you have the possible BLDC commands and the hall sensor readings, the next step is to select which command is used for each reading. The provided code has a **`match hallState:`** section, this will do something different depending on the value of the **`hallState`** variable, it has been set up for you for the 6 possible hall states (1-6 in decimal). Update the **`bldcCmd`** variable in each of the **`cases`** to output the corresponding bldc command depending on the sensor reading as shown in your table. Use the values in the **`bldc`** variable you set up previously. Leave the last **`case _:`** section as is, it should never get triggered.

23. Update the **`enablePhase`** variable after the **`match-case`** section. Depending on the command that is being used, the high and low phases need to be enabled and the floating/off one should be disabled. 1 enables the phase, and 0 disables it.
    Hint: You can use use the **`bldcCmd`** variable as a starting point.

24. Update the **`commandToPhase = 0`** line. The command to the phases should only be given to the phase that is high. Make sure the command amplitude is **`bldcAmplitude`** (0.2) and not 1. This will ensure the voltage given to each phase is not 12V but instead 2V. The **`commandToPhase`** variable needs to have 3 elements: [phase1, phase2, phase3]. Note that the next line in the code clips the values of the variable, which means it will make sure that values stay in the 0 to 1 range.

25. The last lines of the code will print the motor speed once a second, enable the phases based on the **`enablePhase`** variable, update the values that will be written to the bldc

motors based on the **commandToPhase** variable and finally, write those commands to the motor. For now, don't add code in the section for code that runs every 3 seconds.

26. Before running the code, note that clicking on the terminal and using **Ctrl+C** will stop the code. Once the code is running, hold the motor body, if your logic is correct, the motor will feel smooth but vibrating as it turns, if something is wrong in the logic, it will either not rotate, or feel like the motor is not vibrating smoothly, stop your code if you feel this. Run **bldc.py**, the code will run for 30 seconds.

27. The device includes a digital tachometer, which measures the motor speed based on the rate of change of the encoder counts. This value is saved in the **actuators.tach** variable. Once a second, the motor speed will be printed in counts/s.

28. How would you convert your motor speed from counts/s to rads/s or to rpm? Update the code running once a second to update the **rads** and the **rpm** variables and run your code again.

29. Put a piece of tape on the shaft or just gently feel as the shaft is spinning, is it rotating clockwise as expected? If not, what could have gone wrong? Try to fix it if it is not working.

30. What happens if the amplitude of the signal to the motor changes? Try out small changes to **bldcAmplitude**, run the code and notice the differences (test values <=0.3). A 0 to 1 amplitude value corresponds to a 0 to 12V input to the motor.

31. Explore what is the minimum amplitude at which the motor starts turning. What is that in volts? Record the amplitude and the speed at which the motor turns.

32. On the first few lines of the code, change the **frequency** from 400 to 900. Change your **bldcAmplitude** from 0.05 to .8 in 0.05 intervals and run your code for a few seconds each time and record the speed at each amplitude/voltage. Do you get anywhere close to the rated speed of the motor as shown in the motor datasheet?

33. Set the **bldcAmplitude** to 0.4 and run your code at different values of the **frequency**, starting at 900 and going down by 100Hz at a time until you reach 200. What do you observe? How does the motor feel as it is rotating?

34. Bring your amplitude back to 0.15, update your code to try to spin the motor counterclockwise and notice what needs to change.

35. Add code under **if cntr3s == frequency*3:** to change the motor direction every 3 seconds. This means for 3 seconds it will spin clockwise and for 3 seconds it will spin counterclockwise and so on.

36. Save a screenshot of a section of your output printing motor speeds in counts/s and rads/s.

37. Stop, save and close your program.

38. Power OFF the Mechatronic Actuators Trainer, disconnect any motors and pack them along the power supply and USB cables in the provided case.